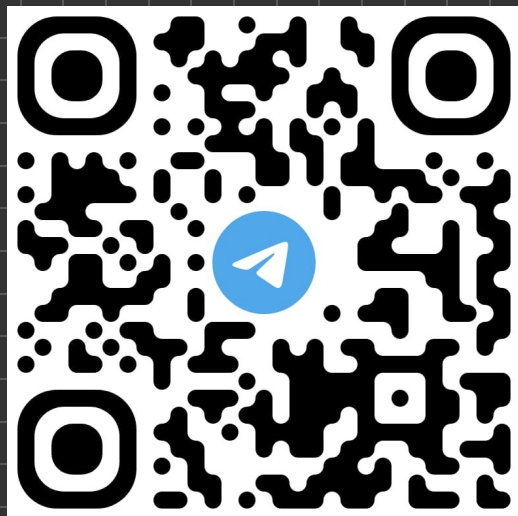


Grid

(Lecture - 17)

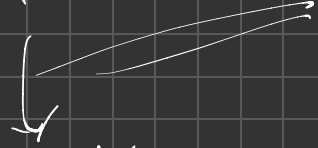
Telegram



Discord



flex box



ID → row | col

↓
layout

Grid
2D → layout
→ col
→ row

CSS Grid is a powerful two-dimensional layout system built natively into CSS that handles both rows and columns simultaneously. Unlike Flexbox, which is primarily one-dimensional, Grid allows you to easily design complex, highly responsive web structures without relying on floats or hacky positioning.

Column

	1	2	3	4	5	6	7
7	1						
6	2						
5	3						
4	4						
3	5						
2	6						
1	7						

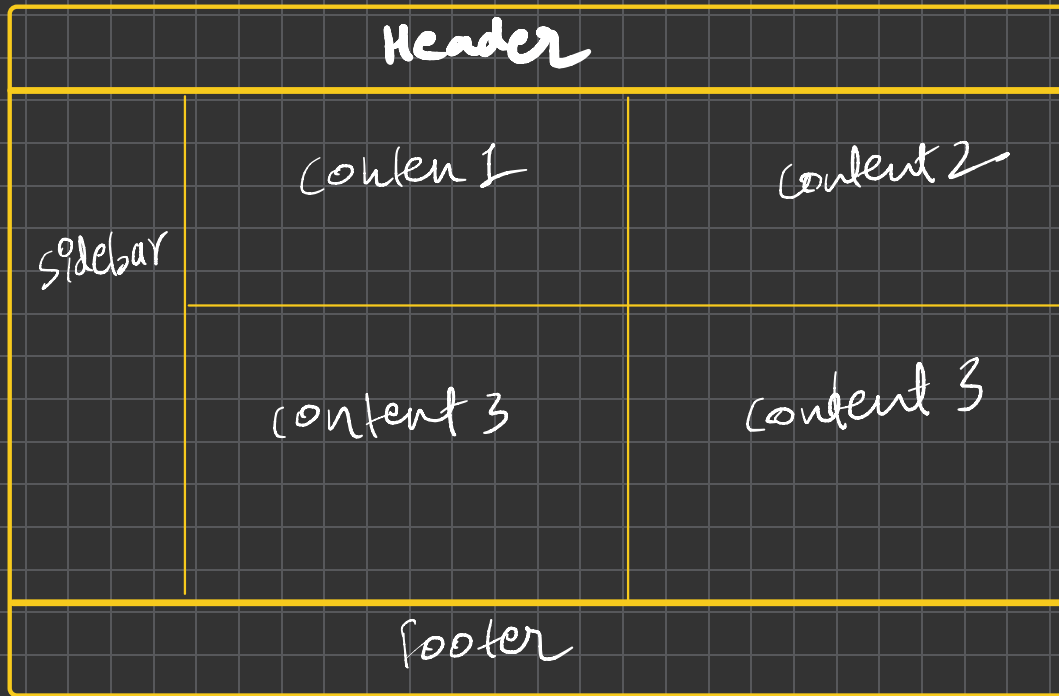
rows

Table

col-lines $\rightarrow$ 7

row-lines $\rightarrow$ 7

row + 1 $\rightarrow$ row lines



mdn

HTMLCSSJavaScriptWeb APIsAllLearnToolsAboutBlog

Web > CSS > Reference > Properties > grid

ThemeEnglish (US)

Filter

font

forced-color-adjust

frame-sizing

gap

> grid-*

grid

hanging-punctuation

height

hyphenate-character

hyphenate-limit-chars

hyphens

> image-*

initial-letter

inline-size

> inset-*

inset

interactivity

> interest-*

interpolate-size

grid CSS property

Baseline Widely available

The **grid** CSS property is a [shorthand property](#) that sets all of the explicit and implicit grid properties in a single declaration.

Using **grid** you specify one axis using [grid-template-rows](#) or [grid-template-columns](#), you then specify how content should auto-repeat in the other axis using the implicit grid properties: [grid-auto-rows](#), [grid-auto-columns](#), and [grid-auto-flow](#).

Try it

CSS Demo: grid

Reset

grid: auto-flow / 1fr 1fr 1fr;

grid: auto-flow dense / 40px 40px

grid: repeat(3, 80px) / auto-flow;

One

Two

Three

In this article

Try it

Constituent properties

Syntax

Formal definition

Formal syntax

Examples

Specifications

Browser compatibility

See also

WETZNER

AD

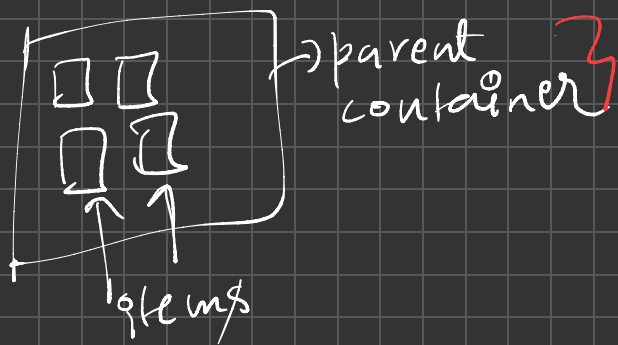
Speedgravity starts with

HOSTING IT YOURSELF

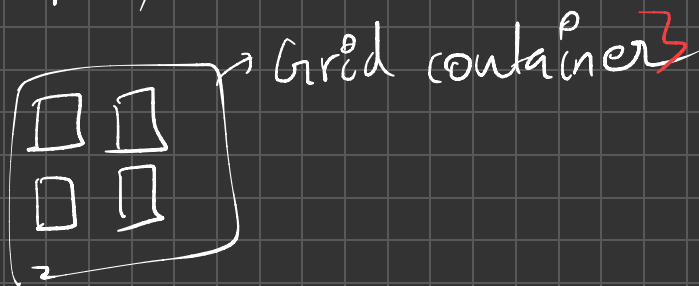
Your project on Hetzner Cloud

Minimize risks, control costs & maximize data protection!

Start now!



child / item



grid-templates - columns



kitne columns?

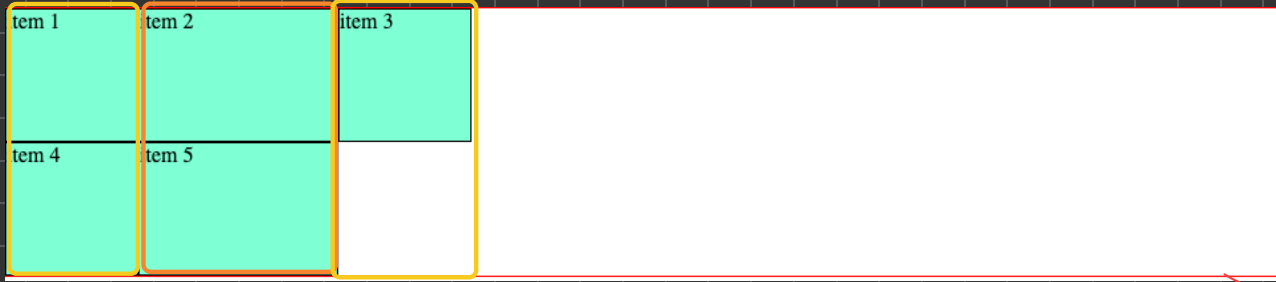
Kya size hoga each column ka

grid-templates - column : 200px 100px 400px —

col 1	col 2	col 3	
col 1	col 2	col 3	

Specifies the size of the columns, and how many columns in a grid layout

`grid-template-columns: 100px 150px 100px;|`



50 vw

← 50% →

item 1 30	item 2 10	item 3 10	↗ 100%
item 4 ← 50%	item 5 →	← 50% →	

← 100% →

↑

flexbox

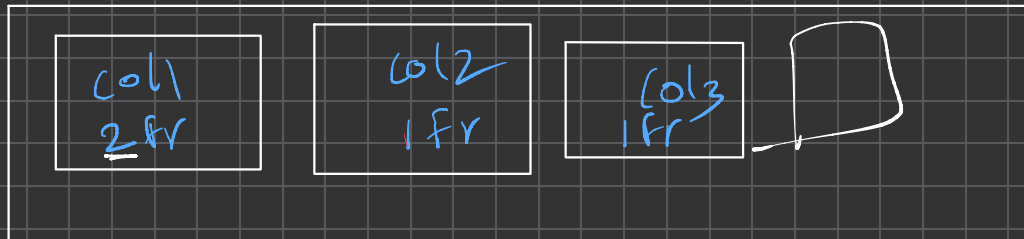
gap

Grid $\rightarrow$ gap

Gap : 10 px $\rightarrow$ row & col

gap : 10 px 20 px
 ↓ ↓
 row col

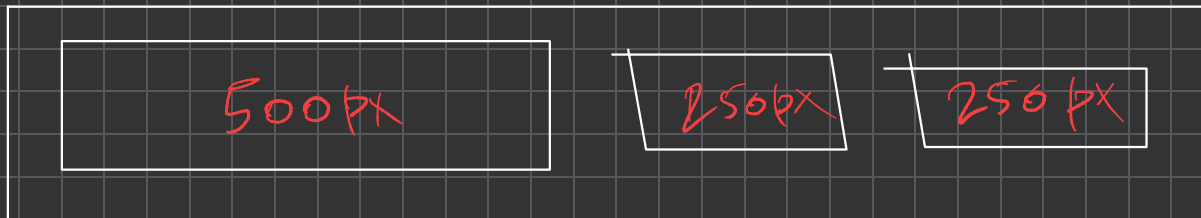
fr → flex-grow



└──────────┬──────────┘
 1000px

4fr

$$\frac{1000}{4} = 250px$$



Repeat function

grid-template-rows : repeat(^{Kitni bar repeat} , [↑] _{size})

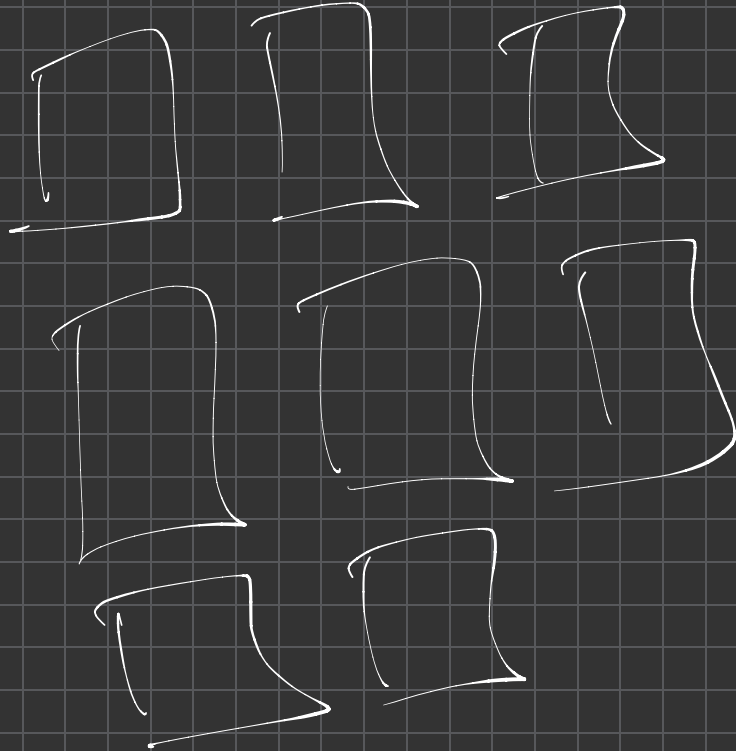
2fr 1fr 1fr
↳ (2 , 1fr)

The repeat() function in CSS Grid is a shorthand utility used to duplicate a row or column track pattern multiple times without rewriting the values manually. It accepts two main arguments: the number of times to repeat the track, and the size of the track(s) being repeated.

500 items

flexbox →

flex-wrap → wrap X

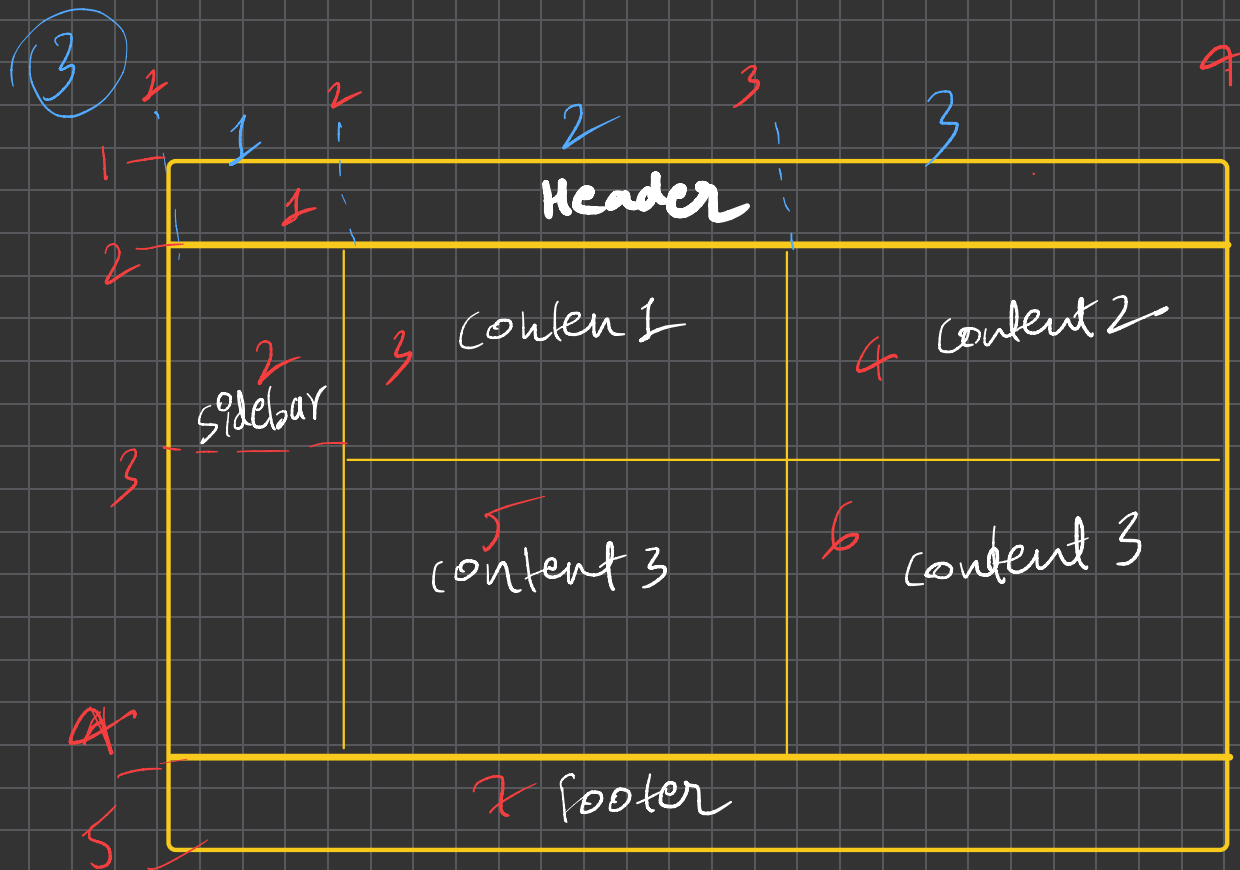


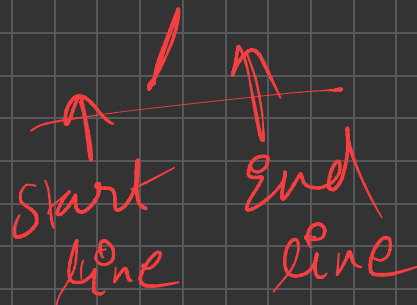
minmax →

minmax (200px, 1fr)
— absolute

5
9:45

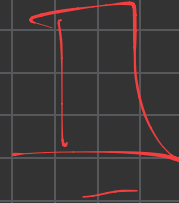
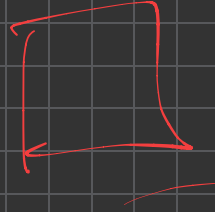
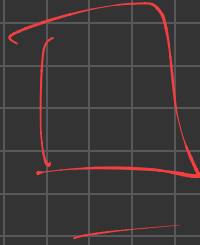
The `minmax()` CSS function defines a size range for CSS Grid tracks, ensuring they stay greater than or equal to a minimum value and less than or equal to a maximum value. It allows you to build highly responsive grid layouts without relying heavily on media queries.



child $\hookrightarrow$ grid-column : 
grid-row : 

grid-column: 1 / 3; (Starts at line 1, ends at line 3, spanning 2 columns).

→ Laptop





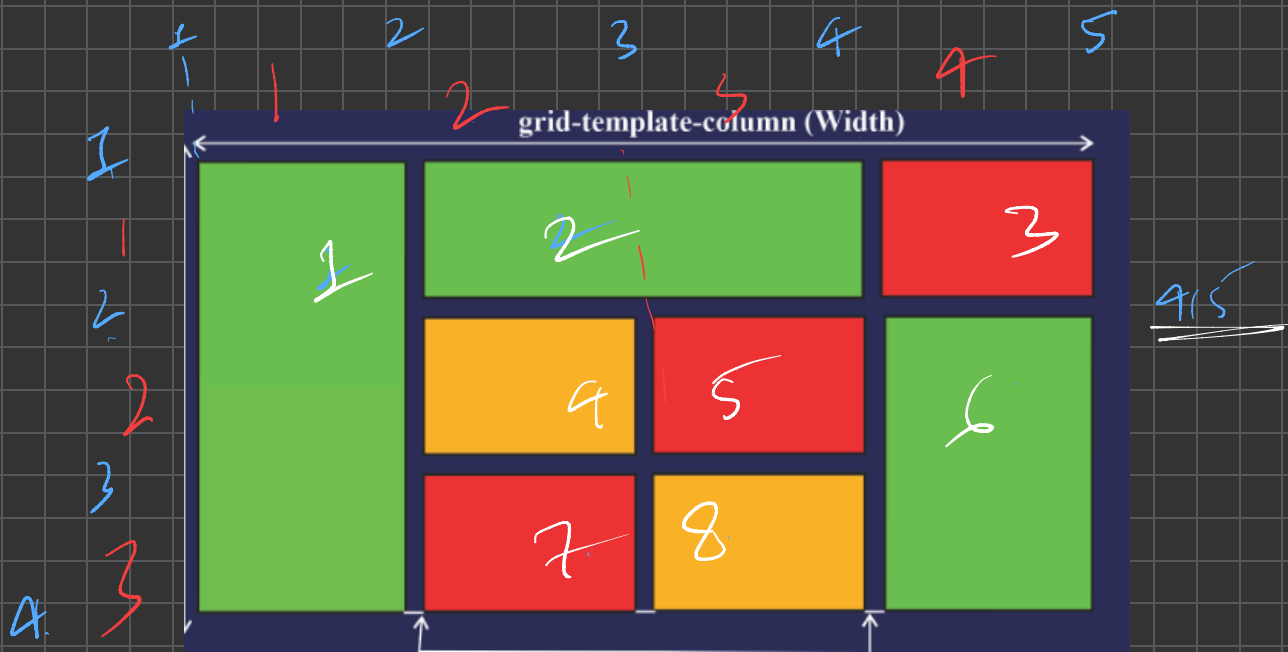
auto-fill

auto-fit

Hu2

- place-items
- place-content
- align-self
- justify-self

- **auto-fill**: Fit as many possible columns as possible on a row, even if they are empty.
- **auto-fit**: Fit whatever columns there are into the space. Prefer expanding columns to fill space rather than empty columns.



HW

